

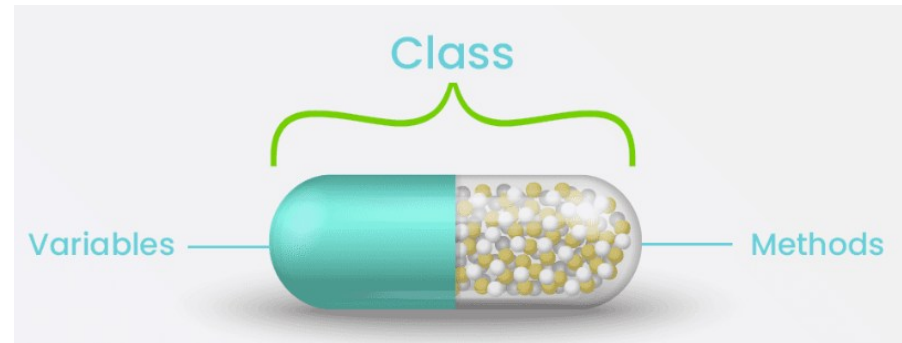
Universitatea Tehnica din Cluj-Napoca
Departament Calculatoare

Programming Techniques in Java

Object-Oriented Design Techniques

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2025-2026

Encapsulation



[Source](#)

Advantages

Hides implementation details by assigning fields and methods the most restrictive visibility.

Mutators allow assigning only valid values to instance variables if validation is implemented.

Accessors and mutators insulate the class from changes to a property's implementation.

Supports thread safety when combined with proper synchronization.

BUT

Getters/setters do not ensure true encapsulation, only controlled access.

Getters/setters can excessively expose internal structure.

Methods may introduce hidden side effects that break predictability.



POORLY ENCAPSULATED CLASSES

Encapsulation

- The issue with getters and setters

```
class BankAccount {  
    public double balance;  
  
    public void update(double value) {  
        balance += value;  
    }  
}
```

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        }  
    }  
}
```

ISSUES

- The getters and setters expose to the outside world that a specific field exists.
- No invariant protection.
- High coupling due to the dependency on getters and setters.
- Race conditions.

```
class Student {  
    private int grade;  
  
    public void setGrade(int grade) {  
        if (grade >= 1 && grade <= 10) {  
            this.grade = grade;  
        }  
    }  
  
    public int getGrade() {  
        return grade;  
    }  
}
```

True encapsulation means that modifications to an object's internal structure should not affect the external entities, except for the public interface.

Encapsulation

- **The issue with methods' side-effects**
 - Side-effect = Any data modification that is observable outside the method
 - The explicit parameter object (i.e., this object)
 - Other objects using explicit parameters, or accessible static fields

In common OO programming practice, all applications need to have side effects because all applications have/manipulate states. However, try to get rid of unnecessary side-effects!

Code with unnecessary side-effects

- Difficult to reason about program behavior, debug
- Difficult to prove program correctness
- Hard to use the code concurrently
- Hard to optimize, test, reuse

Encapsulation is also about controlling *side effects*.

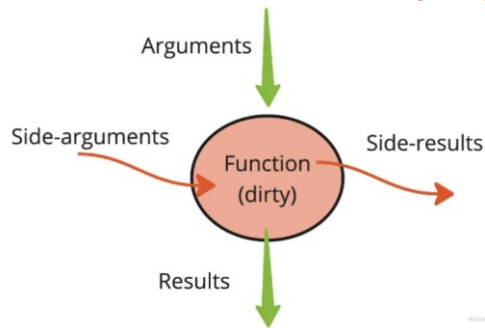
Encapsulation

- **The issue with methods' side-effects**
 - Method without side-effect
 - Always returns the same answer when it is called
 - “pure function” (in math’s terms)
 - Are better composed to form up new functionality
 - Thread safe and could be better debugged
 - Functional programming
 - No side effects
 - No state
 - Immutable variables
 - Favors recursion instead of loops

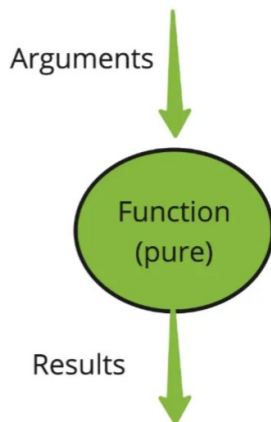
Encapsulation

- Managing side-effects

Method with side-effects (dirty)



Method without side-effects (pure/clean)



```
public int sumWithSideEffects(int a, int b) {  
    Date date = new Date();  
    int result = a + b;  
    System.out.println("At time " + date.toString() +  
        " the result is " + result);  
    return result;  
}
```



```
public int sumPure(int a, int b) {  
    return a+b;  
}
```



[Source](#)

Encapsulation

- **Side-effects as interactions with the outside world**

- Examples

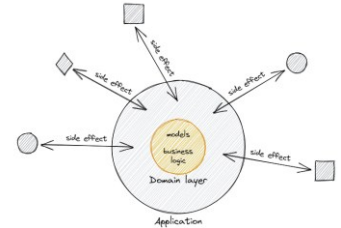
- Reading a file, and/or writing to a file
 - Making a network request (calling an API, downloading a file...)
 - Reading from a global state (e.g. global variable, a parameter from the parent's closure...)
 - Throwing/intercepting an exception
 - Logging messages to the console

- A program with no side effects is not very useful

- We need interactions with the outside world (e.g., getting input from a user, writing to a database, or displaying something to the screen)

- good encapsulation

- Side effects still exist but they are intentional, named clearly and validated



Encapsulation

- **Bad examples**

**Side effect of a method
through accessible static field**

```
public void push() {  
    if (stk.isFull())  
        System.out.println ("Out of Space" );  
}
```

Method with side-effects

```
public class Ex3_NonPure {  
    private int partial = 0;  
    public int sum (int iV) {  
        partial += iV;  
        return partial;  
    }  
}
```

Method with side-effects

```
public class Ex4 {  
    ...  
    public int processAddr(Address adr){  
        ...  
        adr.setStreet(newStreet) { ...}  
    }  
}
```

Encapsulation

- Refactoring code for reducing side-effects
 - Simple approach - example

```
public class FileProcessor {...
    public static void processFile(String filePath) {
        try {
            List<String> lines = Files.readAllLines(Path.of(filePath)); // Side argument
            for (String line : lines) {
                System.out.println(line.toUpperCase()); // Side effect (printing)
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Method with side-effects

```
public class FileProcessor {
    // Step 1: Move side-argument (file reading) up
    public static List<String> readFile(String filePath) throws IOException {
        return Files.readAllLines(Path.of(filePath));
    }
    // Step 2: Convert processing into a pure function
    public static List<String> processLines(List<String> lines) {
        return lines.stream().map(String::toUpperCase).toList();
    }
    // Step 3: Move side-result (printing) down
    public static void displayLines(List<String> processedLines) {
        processedLines.forEach(System.out::println);
    }
}
```

Methods with better managed side-effects

Encapsulation

- **Don't expose your structure - Law of Demeter**
 - A method **should only call methods of**
 - The object itself
 - Its class fields
 - Its parameters
 - Objects it creates
 - A method that follows the Law of Demeter **should NOT**
 - operate on global (non-static) objects or objects that are a part of another object
 - ask another object to give it a part of its internal state to work on (i.e., avoid call getters of other objects)
 - A class should not return a reference to an object that is a part of its internal implementation

```
customer.getOrder().getPayment().process();
```

Encapsulation

- **Law of Demeter - Benefits**

- Simplifies methods by limiting the number of types used inside them
- Restricted operation only on types that you directly have access
- Simplifies changes needed to be done in the future during code extension or maintenance => easier code maintenance

```
double amount = customer.getOrder().getPayment().getAmount();
```

```
class Customer {  
    private Order order;  
  
    public double getPaymentAmount() {  
        return order.getPaymentAmount();  
    }  
}
```

```
class Order {  
    private Payment payment;  
  
    public double getPaymentAmount() {  
        return payment.getAmount();  
    }  
}
```

```
double amount = customer.getPaymentAmount();
```

Encapsulation

- **Law of Demeter - Examples**

```
class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private Engine engine;
    public Car() {
        this.engine = new Engine();
    }
    public Engine getEngine() {
        return engine; // Violation of LoD
    }
}

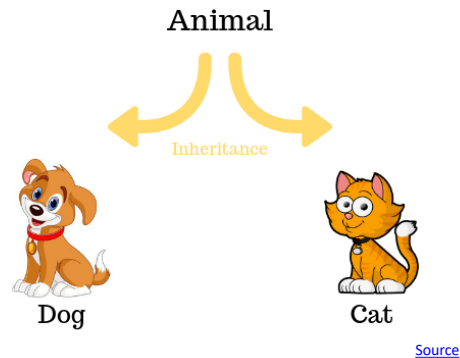
class Driver {
    public static void main(String[] args) {
        Car car = new Car();
        car.getEngine().start(); // Violates LoD
    }
}
```

```
class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private Engine engine;
    public Car() {
        this.engine = new Engine();
    }
    public void startCar() {
        engine.start(); // Encapsulation,
    }
}

class Driver {
    public static void main(String[] args) {
        Car car = new Car();
        car.startCar(); // No direct access to Engine
    }
}
```

Inheritance



Advantages

- Promotes code reuse through shared behavior.
- Models “is-a” relationships (conceptual hierarchy).
- Enables polymorphism.
- Enables the use of an object as its own type or of its base type.
- Reduces duplication in similar classes.
- Supports specialization of behavior.

BUT

Disadvantages

- Introduces tight coupling between parent and child classes.
- Introduces a strong coupling between constructors.
- Fragile base class problem.
- Breaks encapsulation.

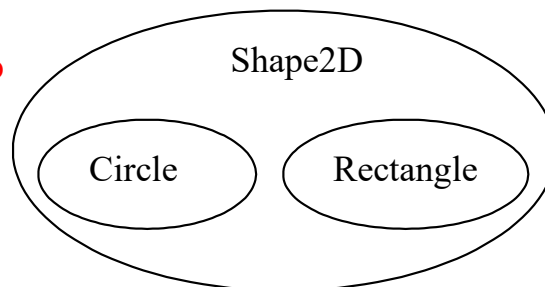


INHERITANCE IS NOT ALWAYS SUITABLE,
FAVOR COMPOSITION!

Inheritance

- Inheritance and subtypes
 - A subclass extends features of its superclass
 - Every instance of the subclass is an instance of the superclass and not vice versa
 - Each class defines a type, and all instances are the set of valid type values
 - The type defined by subclass is valid subtype of the type defined by its superclass
- Subtype relation applies to class types, interface types and primitive types
 - Class inheritance, interface implementation and extension

Is every subclass also a subtype?
Is every subtype a subclass?



- Subclass extends (code relationship)
- Subtype is-a (behavioral relationship)

Inheritance

Condition	Subtype Relationship
If C1 extends C2	C1 is a subtype of C2
If I1 extends I2	I1 is a subtype of I2
If C implements I	C is a subtype of I
For every I	I is a subtype of Object
For every T	T[] is a subtype of Object
If T1 is a subtype of T2	T1[] is a subtype of T2[]
For any C that is not Object	C is a subtype of Object

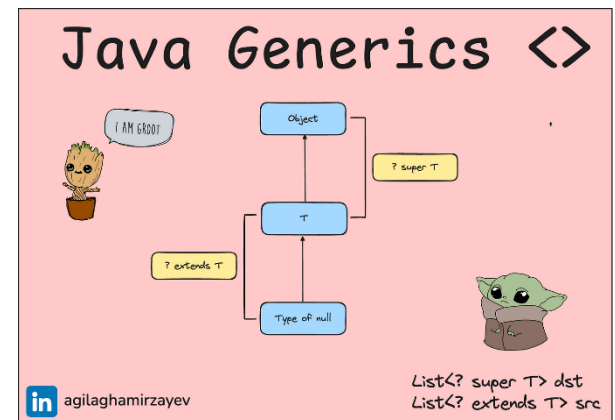
A **subtype** is a class that **can be used in place of another type** without breaking the expected behavior

```
class Animal {}  
class Dog extends Animal {}
```

```
Dog[] dogs = new Dog[3];  
Animal[] animals = dogs;
```

```
Animal[] animals = new Dog[3]; // Allowed at compile
```

```
animals[0] = new Dog(); // Works fine  
animals[1] = new Animal(); // run time error
```



Inheritance

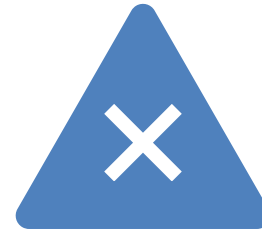


Substitutability of subtypes

A value of a subtype can appear wherever a super-type is expected

In the **context of classes and objects**

- A subclass object can appear wherever a superclass object is expected
- An instance of a subclass can always substitute for an instance of its superclass



Conversion of reference types

Governed by subtype relation

Upcasting

- The conversion of a subtype to one of its supertypes
- Widening is always allowed
- Is carried out implicitly when necessary

Inheritance

- **Conversion of reference types – downcasting techniques**
 - The conversion of a supertype to one of its subtypes
 - Requires explicit casts
 - Allowed at compile time
 - Not always safe – may generate run time exception

Pessimistic approach

```
if(s2 instanceof Circle) {  
    Circle cref = (Circle) s2;  
}  
else {  
    // ... do something else  
}
```

Optimistic approach

```
try {  
    // ...  
    Circle cref = (Circle) s2;  
    // ...  
} catch (ClassCastException e) {  
    // ... do something  
}
```

Examples

```
class Animal { }  
class Dog extends Animal { }  
  
Animal a = new Dog();
```

```
Animal a = new Dog();  
Dog d = (Dog) a;
```

```
Animal a = new Animal();  
Dog d = (Dog) a
```

Reference type is different than the Object type

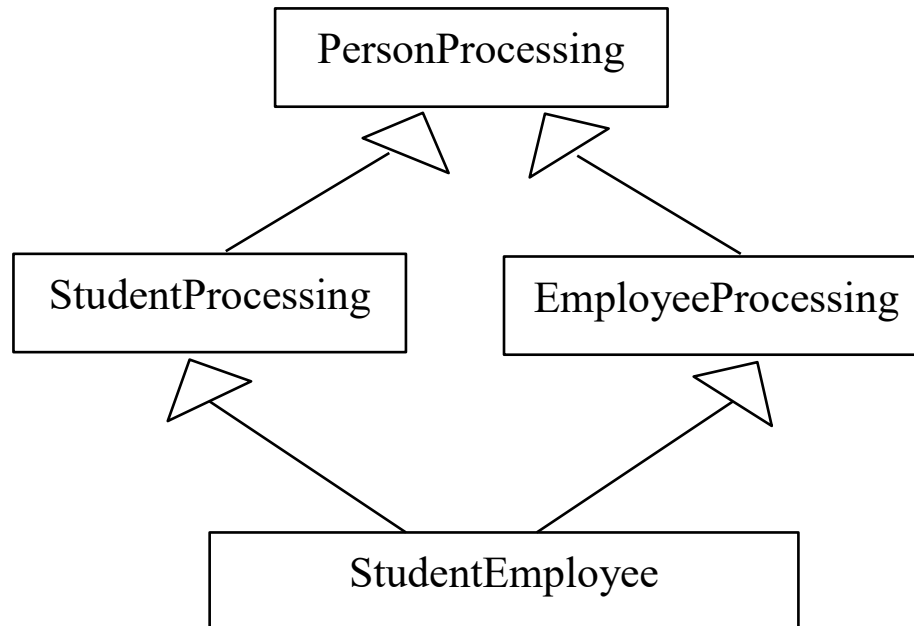
```
Animal a = new Dog();  
  
Reference: Animal  
Object: Dog
```

```
Animal a = new Animal();  
  
Reference: Animal  
Object: Animal
```

Safe way: use instanceof

Inheritance

- **Multiple inheritance issue**
 - C++ feature multiple inheritance
 - Java features simple inheritance
 - Diamond shape inheritance



Inheritance

- Multiple inheritance issue with beyond Java 8 interfaces

```
public interface Interface1 {  
    default void defaultMethod() {  
        System.out.println("Default method in Interface1");  
    }  
}
```

```
public interface Interface2 {  
    default void defaultMethod() {  
        System.out.println("Default method in Interface2");  
    }  
}
```

```
public class ClassImplementingInterfaces implements Interface1, Interface2 {  
    com.compan.y.ClassImplementingInterfaces inherits unrelated defaults for defaultMethod() from types com.compan.y.Interface1 and com.compan.y.Interface2  
    public static void main(String[] args) {  
        ClassImplementingInterfaces aClass = new ClassImplementingInterfaces();  
    }  
}
```



```
public class ClassImplementingInterfaces implements Interface1, Interface2 {  
  
    @Override  
    public void defaultMethod() {  
        Interface1.super.defaultMethod();  
        Interface2.super.defaultMethod();  
    }  
  
    public static void main(String[] args) {  
        ClassImplementingInterfaces aClass = new ClassImplementingInterfaces();  
        aClass.defaultMethod();  
    }  
}
```

Inheritance vs Composition

- **Using inheritance - Stack inherits from ArrayList**

```
public class ArrayList<T> {  
    ...  
    // see if collection is empty  
    public boolean isEmpty() { ... }  
  
    // return size of collection  
    public int size() { ... }  
  
    // add element to the end  
    public void add(T value) { ... }  
  
    // remove element at given index  
    public T remove(int index) { ... }  
  
    // get element from index  
    T get(int index) { ... }  
    ... other class resources  
}
```



```
public class Stack1<T> extends ArrayList<T> {  
    public T push(T elem){  
        T retObject;  
        if(isFull()) retObject = null;  
        else { add(elem); retObject = elem; }  
        return retObject;  
    }  
    public T pop() {  
        T retObject;  
        if(isEmpty()) retObject = null;  
        else {  
            retObject = get(size()-1);  
            remove(size()-1);  
        }  
        return retObject;  
    }  
    public T top() {  
        T retObject;  
        if(isEmpty()) retObject = null;  
        else retObject = get(size()-1);  
        return retObject;  
    }  
    public boolean isFull() { return false };  
}
```

Inheritance vs Composition

- **Using inheritance - Stack inherits from ArrayList**
 - Stack structural component is inherited from ArrayList
 - Adding class specific methods push, pop, top
 - No data elements defined by the class Stack
 - All data elements are inherited from ArrayList
 - Uses the inherited methods in the implementation of the Stack specific methods
 - But inherits all the other methods of ArrayList
 - Problem because now they can be used by Stack objects

```
stack.add(0, string); // Add to bottom of stack  
stack.remove(0);      // Remove from bottom of stack  
stack.indexOf(Object); // Returns the index of the first occurrence
```

Inheritance vs Composition

Advantages of Inheritance

- New implementation is easy, since most of it is inherited
- Less code
- Easy to modify or extend the implementation being reused
- Less overhead in execution, than the composition
- Allows using the new abstraction as an argument in an existing polymorphic method
- Better execution time

Disadvantages of inheritance

- A user (programmer) should study and understand the methods of the superclass
- Operations are more difficult to understand
- Breaks encapsulation, since it exposes a subclass to implementation details of its superclass
- "White-box" reuse, since internal details of superclasses are often visible to subclasses
- Subclasses may have to be changed if the implementation of the superclass changes
- Implementations inherited from superclasses can not be changed at runtime as composition blocks

Inheritance vs Composition

- Using composition - Stack uses ArrayList

The new functionality is obtained by delegating functionality to one of the objects being composed

```
public class Stack2<T> {  
    private ArrayList<T> stk;  
  
    public Stack2() { stk = new ArrayList<>();}  
  
    // behavior  
    public T push(T o) {  
        T retObject;  
        if(isFull()) retObject = null;  
        else {  
            stk.add(o);  
            retObject = o;  
        }  
        return retObject;  
    }  
    public T pop() {  
        T retObject;  
        if(stk.isEmpty()) retObject = null;  
        else { retObject=stk.remove(stk.size()-1);}  
        return retObject;  
    }  
}
```

Inheritance vs Composition

- **Using composition - Stack uses ArrayList**

- Stack class - defines private instance variable (stk) of type ArrayList
- Strong composition when allocating the ArrayList object
- Code reuse
- Difficult work is delegated to ArrayList methods
- Composition makes no explicit or implicit claims for substitutability.
- Stack and ArrayList - entirely distinct entities
- Problems solved

```
stk.add(0, string); // not accessible  
stk.remove(0);      // not accessible
```

Inheritance vs Composition

Advantages of Composition

- Clearly shows all available operations for the abstraction that aggregates
- "Black-box" reuse, => good encapsulation
- Compositions are simple to be changed
- Better separates the two abstractions
- Contained objects are accessed by the containing class solely through their interfaces
- Fewer implementation dependencies
- The composition can be defined dynamically at run-time using polymorphism

Disadvantages of Composition

- Longer code
- Resulting systems tend to have more objects
- Interfaces must be carefully defined in order to use many different objects as composition blocks

Inheritance vs Composition

Always Favor composition over inheritance

- Coad rules for identifying when inheritance should be used**

All
should
be true

R1

A subclass expresses "is a special kind of" and not "is a role played by a"

R2

An instance of a subclass never needs to become an object of another class

R3

A subclass extends, rather than overrides the responsibilities of its superclass

R4

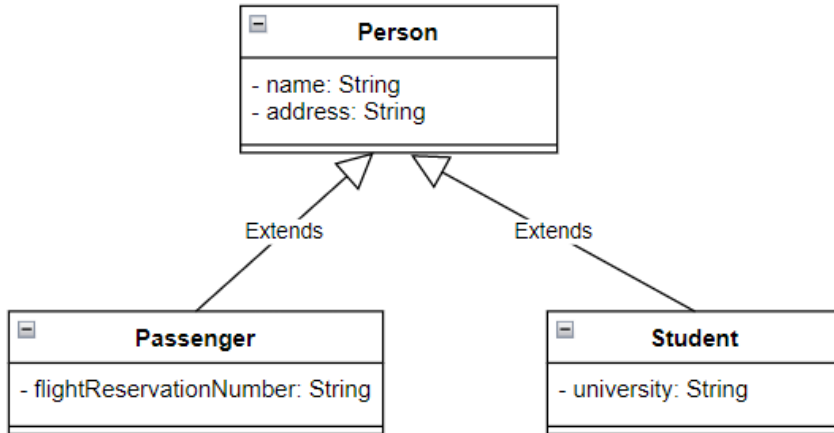
A subclass does not extend the capabilities of what is merely a utility class

R5

For a class in the actual Problem Domain, the subclass specialize a role, action or an entity in the system

Inheritance vs Composition

Coad Rules



Answer to Coad questions:

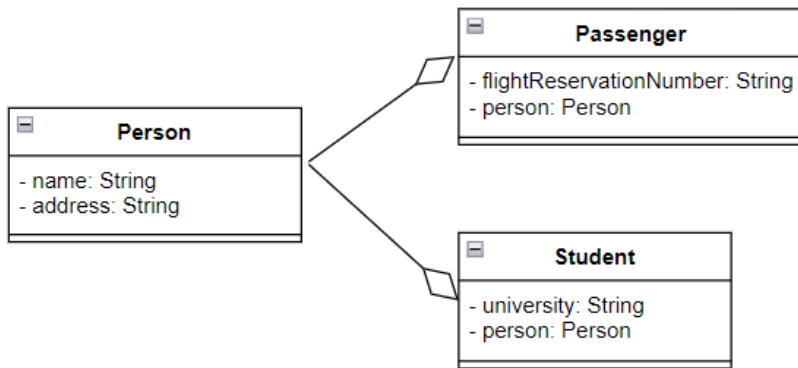
- **Q1:** "is a special kind of" and not "is a role played by a" => **False**
- **Q2:** An instance of a subclass never needs to become an object of another class
 - An instance of a Person may change (in time) from Passenger to Student => **Fail**
- **Q3:** Extends, rather than overrides, the responsibilities of its superclass => **True**
- **Q4:** The subclasses are not extending a utility class => **True**
- **Q5:** For a class in the actual Problem Domain, the subclass specialize a role, transaction or device
=> **False** because a person is not a role, transaction or device

Inheritance is not appropriate here

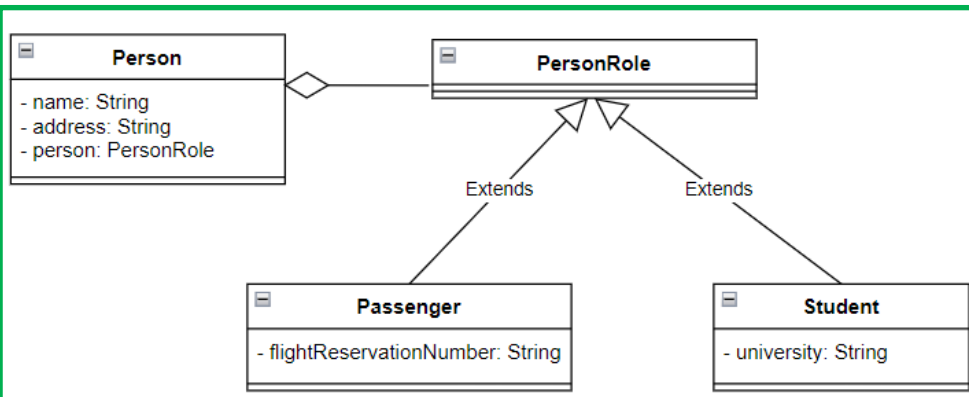
Inheritance vs Composition

Coad Rules

Incomplete solution



Good solution



- **Q1:** Passenger and Student are special kinds of Person roles => **True**
- **Q2:** A Passenger object stays a Passenger object; the same is true for a Student object => **True**
- **Q3:** Extends, rather than overrides or nullifies, the responsibilities of its superclass => **True**
- **Q4:** The subclasses are not extending a utility class => **True**
- **Q5:** For a class in the actual Problem Domain, the subclass specialize a role, transaction or device => **True** because PersonRole is a role

Inheritance is appropriate here

Polymorphism

Advantages

Code Reusability - Write generic code once and apply it to multiple object types.

Flexibility and Extensibility - New classes can be added without modifying existing code.

Loose Coupling - Code depends on abstractions (interfaces), not concrete implementations.

Improved Maintainability - Changes in implementations do not affect client code.

Dynamic Behavior (Runtime Binding) - Behavior is determined at runtime based on the actual object.

Cleaner and Simpler Code - Eliminates conditional logic (if/else, switch) based on type.

Supports the Open/Closed Principle.

Enables Generic Programming - Functions and algorithms work with a wide range of types.

BUT

Disadvantages

Hidden behavior (lack of transparency)

- The actual method executed depends on the runtime type
- Harder to understand what the code really does

Difficult debugging and tracing - must track the concrete object type at runtime

Design complexity

Violation of expectations (Liskov Substitution Principle issues) - Subclasses may not behave consistently with the base type which may lead to subtle bugs

Reduced readability - Indirection hides the real implementation

Polymorphism

Aspect	Compile-time Polymorphism (Method Overloading)	Runtime Polymorphism (Method Overriding)
Binding	Happens at compile-time. <i>Early</i>	Happens at runtime. <i>Late</i>
Method resolution	Based on method signature (parameters).	Based on the actual object type.
Inheritance	Not required. Methods can be in the same class.	Requires inheritance and method overriding.
Example	Method overloading (same method name, different parameters).	Method overriding (same method name, different class implementations).

Polymorphism

- **Method call binding**

- **Early binding** (i.e., static binding) – the binding is performed before the program is run by the compiler
- **Late binding** (i.e., dynamic binding, runtime binding) – the binding is performed at run time based on the type of the object
 - The method call mechanism finds out and calls the correct method body
 - Happens **polymorphically** unless the method is **static** or **final** (**private** methods are implicitly final)

```
public interface Comparable { public int compareTo(Object o); } ✓  
Comparable c = new Date();  
int comp = c.compareTo("Cluj"); ✗
```

Polymorphism

Sub-type	Description
PARAMETRIC	Use generic types that are replaced with concrete types when the code is used. The actual type is determined at compile-time , ensuring type safety.
OVERRIDING	One type can be substituted for another when there is an "is-a" relationship between them. At run-time
OVERLOADING	Using a function with the same name, but different parameters and different code. Compile time
COERCION	The implicit conversion of one data type to another. Automatically when converting from smaller to larger types

Polymorphism

- **Static (Compile Time) Polymorphism**

The *add* method to be invoked is determined at compile time based on the list of parameters

```
public class AddOperation {
    public static int add(int a, int b){
        return a + b;
    }

    public static int add(int a, int b, int c){
        return a + b + c;
    }

    public static void main(String[] args){
        int result1 = AddOperation.add(4, 5);
        int result2 = AddOperation.add(5, 9, 10);
    }
}
```

- **Dynamic (Runtime) Polymorphism**

The *whoAreYou* method is called through a reference of the Shape2D class (parent class) => the method to be executed is determined at runtime based on the type of the assigned object

```
public class Shape2D {
    ...
    public String whoAreYou() {return "SHAPE2D"; }
    ...
}

public class Circle extends Shape2D{
    ...
    public String whoAreYou() { return "CIRCLE"; }
    ...
}

...
Shape2D shape = new Circle();
shape.whoAreYou();
```

Polymorphism

- **Overriding**

- Subclass method has the same elements with the method in the superclass: name, parameters, return type

Overriding with dynamic binding algorithm

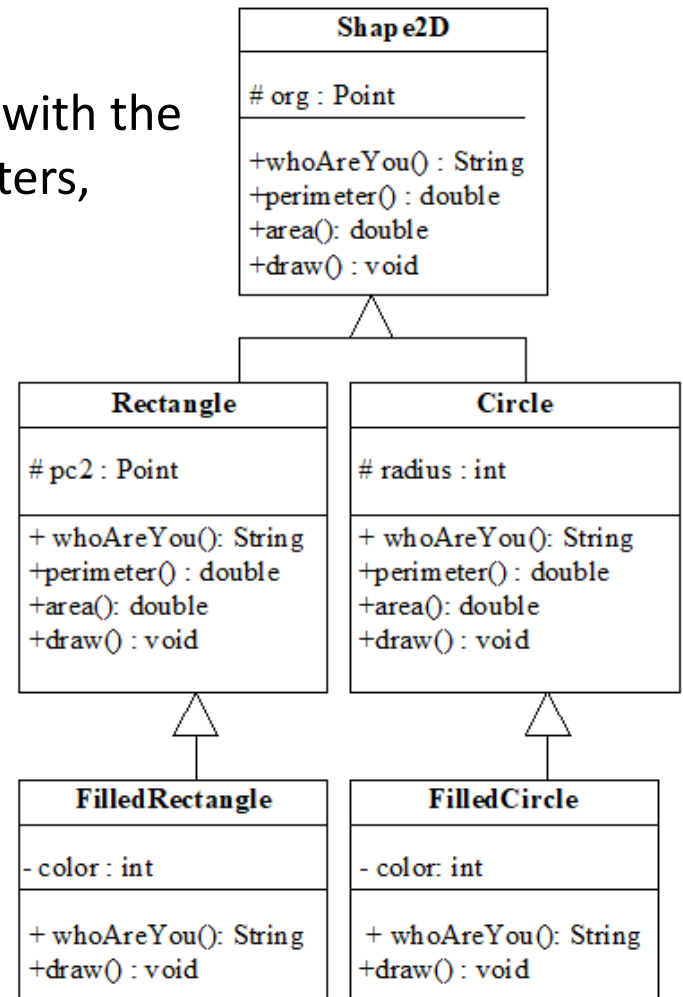
polymorphic method invocation: *obj.m(...)*

crtClass = the class of the object referenced *obj*

repeat

if *m()* is implemented by *crtClass* **then** invoke *m()*

else *crtClass* = the superclass of *crtClass*



Polymorphism

- **Overloading**

- Allows methods of the same class to have the same name and different parameters
- If a Java base class has a method name overloaded several times, redefining that method name in the derived class will not hide any of the base-class versions (unlike C++)

```
public class AClass{  
    public int addNumbers(int n1, int n2){ return n1+n2; }  
  
    public int addNumbers(int n1, int n2, int n3){ return n1+n2+n3; }}
```

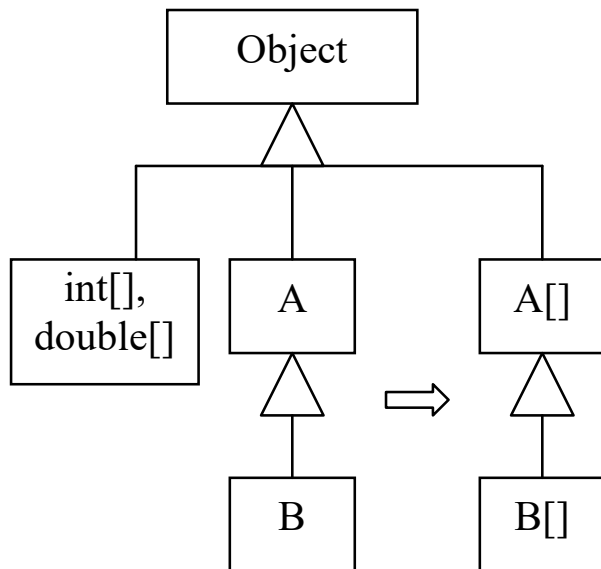
- Overloading on return values – **not possible**

```
public class AClass{  
    public int method1(){ ... }  
  
    public void method1(){ ... }  
}  
...  
method1(); //what method to call?
```

Polymorphism

- **Arrays**

- **Rule 1:** All array types (i.e. `int[]`, `double[]`) are subtypes of `Object`
- **Rule 2:** If class or interface `B` is subtype of class or interface `A` then `B[]` is also subtype of `A[]`



Example of polymorphic assignments in which no explicit casting is necessary

Compile error!!!

Example of polymorphic assignments with explicit downcasting

```
Shape2D sa[];  
Circle ca[] = new Circle[10];  
// ...  
sa = ca;  
Shape2D s1 = sa[2];  
Circle c1 = ca[3];  
  
Circle c2 = sa[0];  
Circle c2 = (Circle)sa[0];
```

Polymorphism

- Polymorphic assignments

```
class Shape2D { ... }  
class Circle extends Shape2D { ...}  
class Rectangle extends Shape2D { ... }
```

```
Shape2D s1, s2;
```

```
s1 = new Circle();  
s2 = new Rectangle();
```

```
Circle c1;
```

```
c1 = s1;
```

```
c1 = (Circle) s1;
```

Example of polymorphic assignments in which no explicit casting is necessary

Compile error!!!:

Even though actually *s1* holds a reference to a *Circle* instance, the declared type of *s1* is *Shape2D* and this is not subtype of left-hand side (i.e. *Circle*)

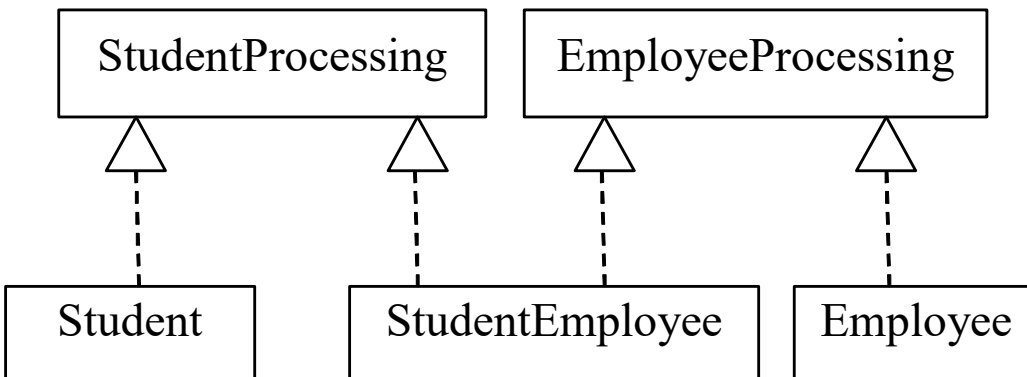
Type checking takes place at compile time

Example of polymorphic assignments with explicit cast

Polymorphism

- **Interfaces and polymorphic classes**

- Interfaces can be used to define polymorphic classes
- A class can implement multiple interfaces => a class shows polymorphic behavior (assumes different roles in different contexts)



```
public interface StudentProcessing {
    public double calculateAvgMarks();
    public double getAvgMarks();
    // ... other methods
}

public interface EmployeeProcessing {
    public double calculateSalary();
    public double getSalary();
    // ... other methods
}
```

Polymorphism

- Interfaces and polymorphic classes

Instances of StudentEmployee seen as student

```
StudentProcessing[] students = new StudentProcessing[SIZE];
students[0] = new Student(...);
students[1] = new StudentEmployee(...);
// ...
for(int i = 0; i < students.length; i++) {
    ... students[i].calculateAvgMarks();
}
```

A student employee
can play two
different roles in two
different contexts

Instances of StudentEmployee seen as employees

```
EmployeeProcessing[] employees = new EmployeeProcessing[SIZE];
employees[0] = new Employee(...);
employees[1] = new StudentEmployee(...);
// ...
for(int i = 0; i < employees.length; i++) {
    ... employees[i].calculateSalary();
}
```

Polymorphism

- Interfaces and polymorphic classes

```
public class Student implements
    StudentProcessing {
    protected double avgMarks;
    public double calculateAvgMarks() {
        // ... method implementation
    }
    public double getAvgMarks() {
        // ... method implementation
    }
}

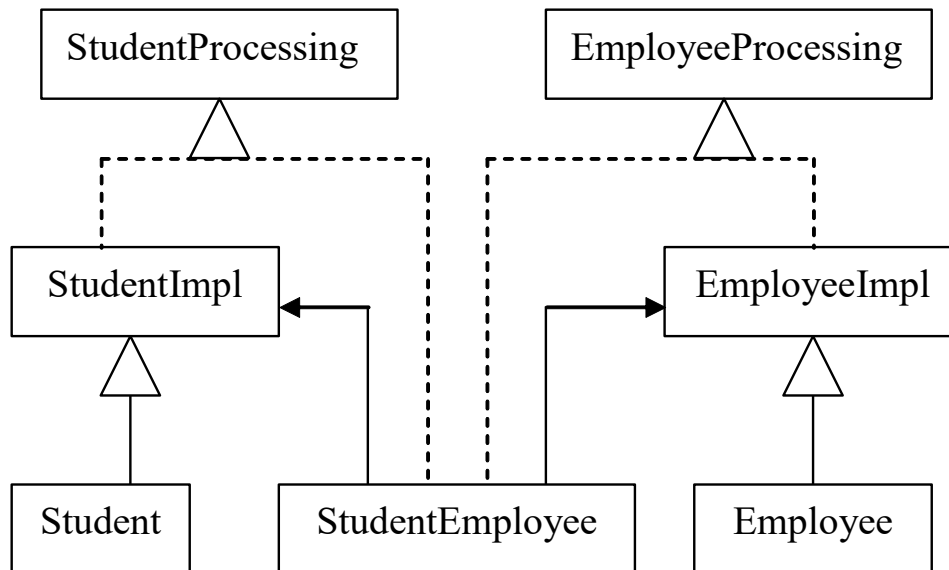
public class Employee implements
    EmployeeProcessing {
    protected double salary;
    public double calculateSalary() {
        // ... method implementation
    }
    public double getSalary() {
        // ... method implementation
    }
}
```

```
public class StudentEmployee implements
    StudentProcessing, EmployeeProcessing {
    protected double avgMarks;
    protected double salary;
    // ... other variables
    public double calculateAvgMarks() {
        // ... method implementation
    }
    public double getAvgMarks() {
        // ... method implementation
    }
    public double calculateSalary() {
        // ... method implementation
    }
    public double getSalary() {
        // ... method implementation
    }
}
```

Sources of inconsistency: there is no common implementation of the methods that are inherited and specialized.

Polymorphism

- Interfaces and polymorphic classes
 - Inconsistency solution: simple inheritance + delegation technique



```
class Delegate {
    // the "delegate"
    void print()
    {
        System.out.println("The Delegate");
    }
}

class Delegator {
    // the "delegator"
    Delegate p = new Delegate ();

    // create the delegate
    void print()
    {
        p.print(); // delegation
    }
}
```

Abstraction Mechanisms

Interfaces versus Abstract Classes

- **Features**

- **Common:** Both define a contract that must be implemented by a class
- Interface inheritance – allows for multiple interface extensions
- A class can implement multiple interfaces
- A class can only extend one other class

```
public interface IF {  
    public void m();  
}
```

```
public abstract class AC {  
    public abstract void m();  
}
```

Abstraction Mechanisms

Interfaces versus Abstract Classes

Able or can do vs. is-a

- Interface
 - Is not describing class main role
 - Describes the peripheral class abilities
 - Example
 - Bicycle may implement Recyclable
 - Many other (unrelated) classes may implement Recyclable
- Abstract class
 - defines the core identity of its descendants
 - Example – class Dog
- Implemented interfaces
 - Specifies what a class can do
 - Doesn't specify what a class is

Abstraction Mechanisms

Interfaces versus Abstract Classes

Plug-in

- Interface
 - New implementations – no common code with previous implementations
 - Start from scratch
 - Freedom to implement a totally new internal design
- Abstract class
 - AC should be used as it is (good or bad)
 - Imposes a certain structure to the new implementer

Third party functionality

- Interface
 - Interface implementation may be added to any existing third-party class
- Abstract class
 - Third-party class must be rewritten to extend from the abstract class

Abstraction Mechanisms

Interfaces versus Abstract Classes

Homogeneity

- Interfaces
 - All the various implementations share is the method signatures
- Abstract class
 - All various implementations are all of a kind and share a common status and behavior

In terms of subclasses

- Abstract class' subclasses are homogeneous
- Interface subclasses are heterogeneous, use interface

Abstraction Mechanisms

Interfaces versus Abstract Classes

Pro interface

- If you think that the API will not change for a long time
- When you need something similar to multiple inheritance

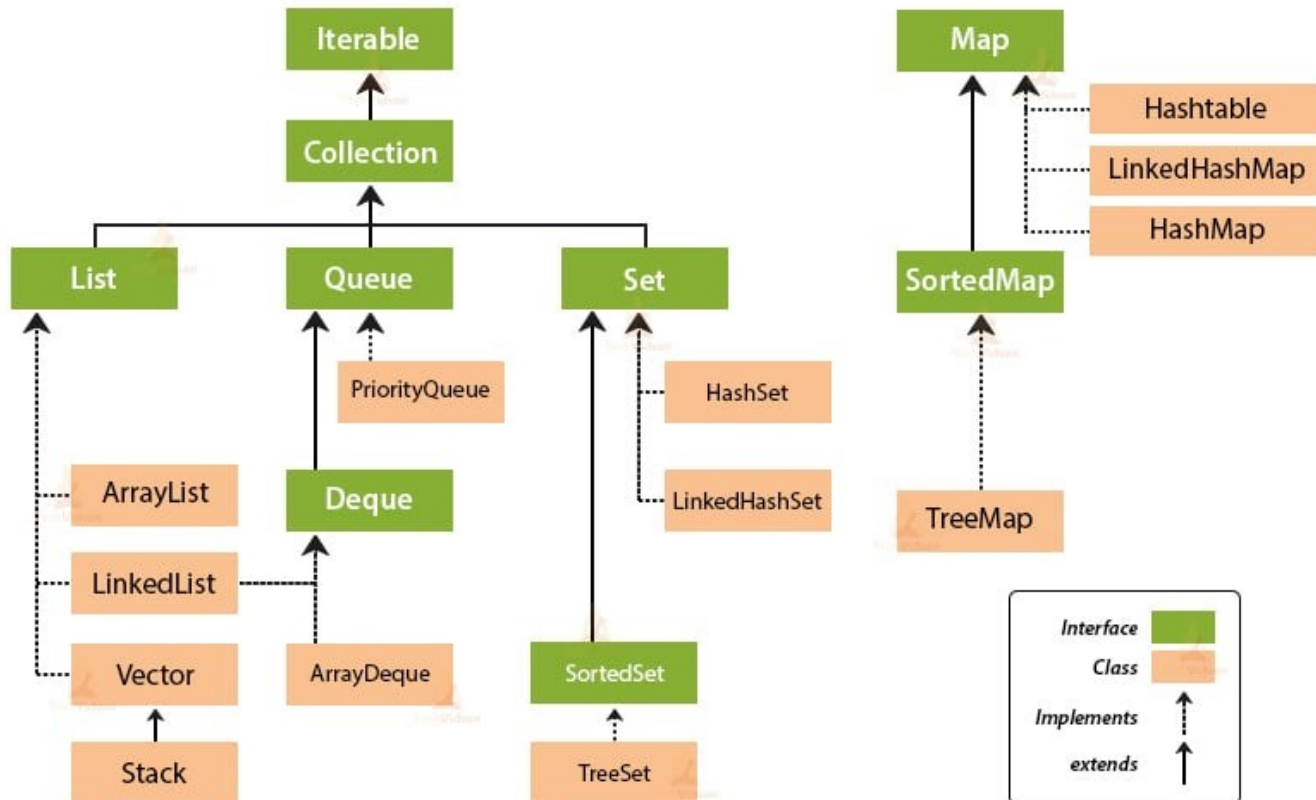
Pro abstract class

- You plan on using inheritance (hierarchies of classes);
- Abstract classes provide a common base class implementation to subclasses

Abstraction Mechanisms

Interfaces versus Abstract Classes

Collection Framework Hierarchy in Java



Abstraction Mechanisms

Interfaces versus Abstract Classes



Level 0 - Top hierarchy

Interfaces such as Collection, List
Describe contracts (behavior specification)



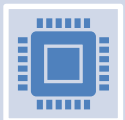
Level 1

Abstract classes such as AbstractList
Provide partial implementations



Level 2

Concrete classes such as ArrayList or Vector
Define all abstract methods that are not already defined



Benefits of programming

Much of the implementation is already done in superclasses
Easy switch between implementations
Develop parallel implementations